
HEAD Bugs: Human-Easy but AI-Difficult Debugging Tasks

Anonymous Authors

Abstract

We study a model-relative regime of debugging tasks that are easy for humans but difficult for code-repair agents. Using a two-layer definition, we first separate tasks by human difficulty and then isolate a canonical HEAD-20 subset inside the human-easy slice via five-seed Mistral Small `act_only` performance. HEAD is supported by five findings. First, the two-layer split exposes a non-trivial human-easy left tail: among 44 human-easy tasks, 20 remain HEAD under the weak-model baseline. Second, the human-difficulty layer is not a loose intuition but a manual rubric over 90 tasks that distinguishes recognizable local bug patterns from tasks requiring deeper algorithmic or stateful reasoning. Third, hint experiments show that localization information partly restores weak-model performance, but the gains are concentrated in QuixBugs, indicating that repair generation remains the broader bottleneck. Fourth, matched protocol–strategy reruns show that plain protocol switching is not enough on HEAD; the best Mistral conditions are `react + early_stop_restart` and `reflexion + self_consistency`, both at 45%. Fifth, after fixing tool-call compatibility, an independent Llama-4-Scout recheck still solves only 8/20 tasks, supporting cross-model robustness of the subset.

1 Introduction

Modern code agents can solve many benchmark tasks, yet they still fail on bugs that human programmers would regard as straightforward. This mismatch suggests that model capability does not align cleanly with human notions of bug difficulty. Rather than treating all failures as belonging to one hard bucket, we ask whether there exists a structured regime of tasks that are easy for humans but disproportionately difficult for agents. This question sits next to recent agentic debugging work built on ReAct, Reflexion, SWE-agent, SWE-bench+, DebugBench, RepairAgent, and InspectCoder [Yao et al., 2023, Shinn et al., 2023, Yang et al., 2024, Jimenez et al., 2024, Wu et al., 2024, Anonymous, 2024, 2025].

We study that regime under the name **HEAD**: *Human-Easy but AI-Difficult*. The key idea is that HEAD is not a claim about intrinsic bug hardness. Instead, it is an operational category defined relative to a debugging setup: a human-easy task counts as HEAD when a weak debugging agent fails on it consistently under a canonical baseline. This lets us separate *definition* from *validation*. Mistral Small under `act_only` provides the definition baseline, while additional experiments probe whether the same subset remains difficult under other prompts, strategies, and model families.

The contribution is empirical and restrained:

1. a two-layer bug classification that isolates a 20-task canonical HEAD subset;
2. a hint study showing that localization helps but does not fully explain HEAD;
3. a matched protocol–strategy study showing that trajectory control matters more than plain protocol switching on Mistral HEAD20;
4. a Reflexion ablation showing no stable aggregate gain over ReAct in the matched batch;

Table 1: Experimental design summarized by role rather than by chronological run order.

Layer	Scope	Models / conditions	Purpose
Human-difficulty annotation	90 tasks	manual rubric	separate human-easy from
AI-difficulty definition	44 human-easy tasks	Mistral Small, <code>act_only</code> , 5 seeds	define Easy / Intermediate
Hint study	HEAD20	Mistral L0/L2/L4, 120B L0/L2	test localization sensitivity
Matched protocol study	HEAD20	3 protocols $\times$ 4 strategies, shared budgets	compare trajectory-control
Mechanism ablation	HEAD20	ReAct / Reflexion matched reruns	test whether Episode 2 d
Cross-family recheck	HEAD20	Llama-4-Scout, <code>act_only</code> baseline	probe transfer beyond M

5. an independent Llama-4-Scout recheck showing that the Mistral-defined subset remains difficult for another model family.

2 Task Construction and Experimental Setup

2.1 Benchmarks

The study contains 90 debugging tasks drawn from three sources:

- **QuixBugs**: 40 algorithmic program-repair tasks;
- **Mini-nightmare**: 10 compact, realistic debugging tasks;
- **DebugBench**: 40 debugging tasks spanning reference, syntax, logic, and multiple-error patterns [Wu et al., 2024].

The experiments use a single local debugging-agent setup across all reported conditions so that task difficulty, prompting, and model family can be compared under a shared environment.

2.2 Two-Layer Difficulty Definition

The first layer separates tasks by human difficulty using manual task annotations over all 90 tasks. The annotation rubric is intentionally operational rather than psychological. A task is labeled `human-easy` when a competent programmer can find and repair the bug by recognizing a familiar error pattern directly from the code and test output, without reconstructing the surrounding algorithm in depth or tracing a long chain of state changes. Typical cases include syntax errors, typos, wrong variable or attribute names, obvious operator mistakes, simple off-by-one errors whose direction is revealed by the failing output, and missing returns or guard conditions that are already implied by the function contract.

A task is labeled `human-difficult` when repairing it plausibly requires deeper algorithmic understanding, extended control-flow tracing, domain-specific reasoning, concurrency or mutable-state analysis, or untangling multiple interacting errors. The annotation records not only the binary label but also a short reasoning field, a coarse bug-pattern description, and the approximate fix region. The purpose of these annotations is to anchor the first layer in explicit repair criteria before any model outcomes are considered.

The second layer refines only the `human-easy` slice using five-seed Mistral Small `act_only` pass counts:

- **Easy**: at least 3/5 passes;
- **Intermediate**: exactly 2/5 passes;
- **HEAD**: at most 1/5 passes.

This design makes HEAD a high-precision operational subset rather than a universal theory of AI difficulty.

2.3 Models and Conditions

The model roles are:

Table 2: Two-layer classification used in this study.

Slice	Count
human-easy	44
human-difficult	46
human-easy / Easy	16
human-easy / Intermediate	8
human-easy / HEAD	20

Table 3: Composition of the canonical HEAD20 subset by benchmark family.

Family	Count
DebugBench	7
Mini-nightmare	4
QuixBugs	9

- **Definition model:** `api-mistral-small-3.2-2506`;
- **Strong reference:** `api-gpt-oss-120b` in the hint study;
- **Independent recheck:** `api-llama-4-scout`.

The main protocol families are `act_only`, `react`, and `reflexion`. The matched Mistral rerun compares 12 protocol–strategy conditions under a shared seed, token budget, and turn budget.

3 Main Results

The results are organized as a progression rather than as a flat list. We first establish that HEAD is a real slice inside the human-easy region and then strengthen that definition with an external recheck on a different model family. We next ask what kind of difficulty this slice reflects, using hints and trajectory aggregates as diagnosis tools. Finally, we test which interventions help once the slice has been isolated.

3.1 Existence: HEAD forms a real left tail inside human-easy tasks

Table 2 shows the first main result: human-easy does not imply agent-easy. Among 44 tasks judged easy for humans, only 16 are easy for the Mistral baseline, 8 lie in an intermediate band, and 20 fall into the HEAD subset. This means nearly half of the human-easy slice still looks difficult to the weak agent under the canonical setup.

This result is important because it separates HEAD from generic benchmark hardness. HEAD is not built from human-difficult tasks. It is the left tail *inside* the human-easy region. Table 3 further shows that the canonical subset is not tied to a single benchmark source: it contains 7 DebugBench tasks, 4 Mini-nightmare tasks, and 9 QuixBugs tasks. The phenomenon therefore survives across both algorithmic repair tasks and compact debugging tasks with more open-ended execution traces.

3.2 External recheck: the Mistral-defined subset remains difficult for Scout

The definition itself is Mistral-relative, so it is important to ask whether the resulting subset is still difficult for another model family. We therefore run an independent recheck with `api-llama-4-scout`. After extending the runner to accept the model’s bracketed pseudo-tool-call format, Scout solves only 8/20 tasks under `act_only` + `baseline`.

This recheck does not redefine HEAD; Mistral remains the definition model. It does, however, strengthen the interpretation of HEAD as a model-relative regime rather than a one-model accident. In particular, the complete failure on Mini-nightmare suggests that the hardest part of HEAD20 is not just an artifact of one model API or one prompt family. The task-level list in the appendix is consistent with the family totals in Table 4: five of the eight Scout successes come from QuixBugs, three come from DebugBench, and none come from Mini-nightmare.

Table 4: Independent Llama-4-Scout recheck on HEAD20.

Family	Pass rate	Avg. tokens
Overall	8/20 (40.0%)	36,260
DebugBench	3/7 (42.9%)	35,038
Mini-nightmare	0/4 (0.0%)	53,540
QuixBugs	5/9 (55.6%)	29,531

Table 5: Hint response on the canonical HEAD20 subset.

Hint level	Mistral pass	120B pass
L0 (no hint)	1/20 (5%)	14/20 (70%)
L2 (function name)	5/20 (25%)	20/20 (100%)
L4 (line-level hint)	8/20 (40%)	—

3.3 Diagnosis: HEAD combines localization sensitivity with a convergence bottleneck

3.3.1 Hints help, but their benefits are uneven

The hint study uses `react` baseline runs with increasing localization information. The full current coverage includes all 20 HEAD tasks for Mistral at levels L0/L2/L4 and for 120B at L0/L2.

Table 5 supports two claims. First, localization information clearly matters: Mistral improves from 5% to 25% with function-level hints and to 40% with line-level hints. Second, localization is not the whole story. The Mistral gains are concentrated in QuixBugs, while DebugBench and Mini-nightmare remain mostly unsolved. At the family level, reported in Appendix Table 11, QuixBugs improves from 1/9 to 4/9 to 7/9 as hints become more specific, whereas DebugBench moves only from 0/7 to 1/7 to 1/7 and Mini-nightmare remains at 0/4 throughout. This asymmetry suggests that HEAD is not a single failure type. Some tasks mainly need sharper localization; others remain difficult even after the search region has been narrowed, which points more directly to repair synthesis and validation as the limiting stage.

For the strong reference model, 120B already solves 70% of HEAD20 without hints and reaches 100% with function-level hints. In this regime, hints behave more like context sharpening than like a rescue mechanism. Taken together, these results diagnose only part of the problem: some HEAD tasks are localization-sensitive, but the family-level asymmetry already suggests that localization cannot be the full explanation.

3.3.2 Trajectory aggregates indicate failure to converge rather than failure to start

Table 6 shows that weak-model failures on HEAD are not primarily delayed-activation failures. Across all three baseline protocols, both models begin editing at roughly the same point: the first edit arrives around turn 5 for both Mistral and 120B. The more striking gap appears after editing begins. On `act_only`, 120B averages 2.9 patch attempts and 2.5 test runs while solving 15/20 tasks, whereas Mistral averages 5.4 patch attempts and 3.4 test runs while solving only 4/20. The same qualitative gap remains visible for `react` and `reflexion`: Mistral continues to consume more patch attempts and test executions per task than the stronger model, despite similar first-edit timing.

These aggregates imply that HEAD is not mainly a regime where the weak model fails to locate a starting point. Instead, the weak model often reaches an editable region, begins modifying code, and still fails to convert those edits into a correct fix. For the stronger model, the edit-to-fix gap stays in a fairly stable 3.6–4.0 turn band across protocols; the weaker model often reaches the edit phase but rarely turns that progress into a reliable fix, especially under `react`. The cost signature is therefore extended unsuccessful search rather than slow activation. This observation complements the hint study: better localization helps some tasks, but even precise hints do not remove the broader convergence problem.

Table 6: Aggregate baseline trajectory metrics on HEAD20.

Model	Protocol	Pass	First edit	First fix	Edit→fix gap	Patch attempts	Test runs
120B	act_only	15/20	5.3	8.7	3.6	2.9	2.5
120B	react	14/20	5.5	9.1	3.9	2.8	2.6
120B	reflexion	16/20	5.5	9.7	4.0	2.4	2.7
Mistral	act_only	4/20	5.2	8.2	3.0	5.4	3.4
Mistral	react	1/20	5.3	8.0	—	4.8	3.5
Mistral	reflexion	7/20	5.1	8.9	3.3	5.0	3.5

Table 7: Best-performing matched Mistral conditions on HEAD20.

Condition	Pass rate	Avg. tokens
react + early_stop_restart	9/20 (45%)	36,770
reflexion + self_consistency	9/20 (45%)	37,288
reflexion + early_stop_restart	8/20 (40%)	39,143
react + self_consistency	8/20 (40%)	42,329
act_only + baseline	5/20 (25%)	41,638

3.4 Intervention: protocol–strategy combinations help by improving convergence, not by adding protocol complexity alone

Section 3.2 showed that HEAD is not well described as a failure to begin acting. The weak model usually reaches an editable region and starts making changes, but then fails to convert those attempts into a correct fix. The next question is therefore not simply which protocol is “stronger,” but which intervention most effectively improves convergence once a run is already underway. The evidence in this section points to a consistent answer: the best gains come from protocol–strategy combinations that restructure how search unfolds, especially by shortening unproductive trajectories or redistributing budget across independent attempts.

3.4.1 Protocol switching alone is not enough on HEAD

The matched Mistral rerun compares 12 protocol–strategy conditions on HEAD20. The key takeaway is that the best gains do not come from plain protocol upgrades by themselves. They come from protocol–strategy combinations that reduce the cost of staying on a bad trajectory.

Two observations follow from Tables 7 and 8. First, plain protocol switching is helpful but limited: the baseline progression from act_only to react to reflexion is only 25% → 35% → 30%. Second, the largest gains appear only when a protocol is paired with a particular strategy. Within-protocol deltas are consistently larger than the baseline protocol gaps: react + early_stop_restart improves over react + baseline by 10 points, and reflexion + self_consistency improves over reflexion + baseline by 15 points. Relative to the convergence diagnosis in Section 3.2, this pattern is informative: the winning interventions are precisely the ones that reduce the cost of staying on a bad path, either by terminating it earlier or by allocating budget across multiple shorter attempts.

The broader ranking in Appendix Table 12 reinforces this interpretation. Plain baseline ordering is modestly separated—act_only at 25%, reflexion at 30%, and react at 35%—but none of these baseline moves is decisive on its own. The strongest conditions are the paired interventions that either cut off unproductive trajectories early (early_stop_restart) or allocate budget across multiple independent attempts (self_consistency). The bottom of the ranking is also informative: reflexion + checklist falls to 20% despite using the heaviest protocol, which argues against a simple “more reasoning is better” interpretation. On this subset, weak-model performance depends more on how the run budget is structured than on protocol label alone.

3.4.2 Reflexion does not show a stable aggregate edge in the matched ablation

We also include a matched Reflexion mechanism ablation on HEAD20. All conditions were rerun in one batch to avoid comparing a new ablation to older baselines.

Table 8: Improvement over the same-protocol baseline on HEAD20.

Protocol / strategy	Pass rate	Delta vs. baseline
act_only + baseline	5/20 (25%)	—
act_only + checklist	7/20 (35%)	+10 points
act_only + early_stop_restart	7/20 (35%)	+10 points
react + baseline	7/20 (35%)	—
react + self_consistency	8/20 (40%)	+5 points
react + early_stop_restart	9/20 (45%)	+10 points
reflexion + baseline	6/20 (30%)	—
reflexion + early_stop_restart	8/20 (40%)	+10 points
reflexion + self_consistency	9/20 (45%)	+15 points

Table 9: Reflexion mechanism ablation on HEAD20.

Condition	Pass rate	Avg. tokens
react_baseline	8/20 (40%)	38,306
reflexion_baseline	8/20 (40%)	37,230
reflexion_ep1_only	7/20 (35%)	37,813
restart_without_reflection	5/20 (25%)	46,284

The ablation does not reproduce a clear aggregate Reflexion-over-ReAct advantage. Reflexion baseline ties ReAct baseline at 8/20, and the family breakdown is also identical at 1/7 DebugBench, 0/4 Mini-nightmare, and 7/9 QuixBugs. The two conditions exchange only one QuixBugs success in either direction. Likewise, `reflexion_ep1_only` reaches 7/20, only one task behind ReAct, so ReAct does not behave as a strict lower bound on Reflexion Episode 1. Most importantly, neither two-episode condition produces an `Ep1 fail` $\rightarrow$ `Ep2 pass` rescue. Meanwhile, clean restart without reflection is both weaker and more expensive. The most conservative interpretation is therefore negative: in the current matched batch, Episode 2 is not a demonstrated positive contributor, and naive restart alone is not the source of the gains seen in Table 8.

This is substantively different from saying that Reflexion never helps. The narrower claim supported here is that the observed gains on HEAD20 are better explained by convergence management than by a stable second-episode rescue mechanism. Appendix Table 13 and its accompanying readouts show the same pattern at slightly finer granularity: no Episode-2 rescues are observed, and `restart_without_reflection` drops to 25% while consuming the most tokens. The evidence therefore supports a specific reading of Section 3.3 as a whole. On HEAD, the useful interventions are not simply those that add another reasoning stage; they are the ones that prevent the agent from spending too much of its budget on a single failing trajectory.

4 Discussion

Taken together, the results support a more specific interpretation of HEAD than a generic “hard bug” label.

First, HEAD is best understood as a model-relative regime inside the human-easy region. The first layer of annotation rules out the trivial explanation that these tasks are simply hard for everyone. The second layer then isolates the subset on which a weak model fails consistently under a fixed baseline. The Scout recheck does not change the definition, but it shows that the Mistral-defined subset still transfers non-trivially to another family. This makes HEAD more than a single-model artifact while still keeping the definition operational and narrow.

Second, the experiments suggest that the central difficulty in HEAD is not pure localization failure. The hint study shows that localization information matters, but the gains are highly uneven: QuixBugs improves sharply under stronger hints, while DebugBench and Mini-nightmare remain largely resistant. The trajectory aggregates sharpen this point. In all three baseline protocols, the weak model begins editing at roughly the same time as the stronger model, yet it converts those attempts into

correct fixes much less reliably. The most coherent reading is therefore that many HEAD failures arise after the agent has already engaged with the right region, during the process of converging on a correct repair and validating it.

Third, the intervention results narrow what kinds of help currently appear effective. The matched protocol study does not show a dominant protocol in isolation; the baseline protocol gaps are modest, and Reflexion baseline does not separate from ReAct in the matched ablation. What does stand out is that the strongest gains come from specific protocol–strategy combinations such as `react + early_stop_restart` and `reflexion + self_consistency`. At the same time, the ablation rules out an overly simple explanation: adding a second episode or restarting naively is not, by itself, a reliable source of improvement. The practical implication is that HEAD currently behaves more like a search-control problem than a protocol-selection problem. Weak models benefit most when the run is prevented from overcommitting to a single failing trajectory.

These conclusions remain intentionally bounded. The present evidence is sufficient to argue that HEAD is a meaningful and partially transferable regime, and that its observed failures are more consistent with post-edit convergence problems than with failure to start. It is not yet sufficient to claim a complete taxonomy of failure mechanisms. That stronger claim would require direct trajectory annotation, broader multi-seed cross-model analysis, and subtype-level intervention studies.

5 Limitations and Future Work

The paper has four main limitations.

- The canonical HEAD definition is tied to one weak model family and one protocol baseline.
- Some results, especially the Scout recheck, are currently single-seed.
- The current evidence is mostly aggregate; it shows *that* HEAD exists, but not yet a full trajectory-level account of *why* each subtype fails.
- Strong-model evidence is narrower than the weak-model evidence and is mostly visible through the hint study.

The most valuable next steps are multi-seed cross-model validation, subtype-level analysis inside HEAD20, and manual trajectory annotation that distinguishes localization, repair, and search-control failures.

6 Conclusion

Within the scope of this study, HEAD is already a useful empirical object. A two-layer definition isolates a 20-task subset of human-easy bugs that remain difficult for a weak debugging agent. The human-difficulty layer makes explicit that these are not generic hard tasks: they are tasks whose fixes remain locally legible to a programmer, yet still frustrate the agent. Hint experiments show that localization helps but does not eliminate the phenomenon. Matched reruns show that the best improvements come from controlling bad trajectories rather than simply changing protocol names. Reflexion ablations weaken the claim that Episode 2 is a stable source of improvement. Finally, a Scout recheck shows that the subset remains difficult for an independent model family.

A Two-Layer Definition Details

The canonical HEAD definition uses five-seed Mistral Small `act_only` performance inside the human-easy slice. The thresholds are repeated here for completeness.

This definition is narrow on purpose. It is intended to produce a stable high-precision subset rather than a universal definition of AI debugging difficulty.

B Hint Study Details

Line-level hints mainly improve QuixBugs. DebugBench and Mini-nightmare remain largely unsolved even after the search region is narrowed.

Table 10: Operational definition of the second-layer categories inside human-easy.

Category	Rule
Easy	at least 3/5 Mistral <code>act_only</code> passes
Intermediate	exactly 2/5 passes
HEAD	at most 1/5 passes

Table 11: Family-level Mistral hint response on HEAD20 under `react` baseline.

Family	L0	L2	L4
DebugBench	0/7	1/7	1/7
Mini-nightmare	0/4	0/4	0/4
QuixBugs	1/9	4/9	7/9

C Matched Mistral Protocol-Strategy Table

Mini-nightmare is omitted from the last two columns because it is almost entirely zero in this matched rerun; only `act_only` + `checkList` solves one of the four tasks. Most of the variation comes from QuixBugs and, to a lesser extent, DebugBench.

D Reflexion Mechanism Details

Additional readouts:

- Episode-1 pass rate: 7/20 (35%).
- Episode-2 rescues in baseline: 0.
- Episode-2 rescues in restart-without-reflection: 0.
- Baseline success but restart-without-reflection fail: 5 tasks.
- Baseline success but ReAct fail: 1 task.
- ReAct success but baseline fail: 1 task.

No matched condition in this ablation shows a stable Episode-2 rescue effect.

E Independent Scout Recheck Details

E.1 Compatibility Note

Scout initially emitted bracketed pseudo-tool calls such as `[run_tests()]` rather than formal API tool calls. The fallback parser was then extended to accept this format, and only the post-fix rerun is treated as valid evidence.

E.2 Task-Level Summary

The eight successful Scout tasks are:

- `debugbench_012_corporate_flight_bookings`
- `debugbench_031_next_greater_element_i`
- `debugbench_037_minimum_bit_flips_to_convert_number`
- `quixbugs_is_valid_parenthesization`
- `quixbugs_mergesort`
- `quixbugs_quicksort`
- `quixbugs_to_base`

Table 12: Full matched Mistral protocol-strategy ranking on HEAD20.

Protocol	Strategy	Pass	Pass %	Avg. tokens	DebugBench	QuixBugs
react	early_stop_restart	9/20	45.0	36,770	2/7	7/9
reflexion	self_consistency	9/20	45.0	37,288	2/7	7/9
reflexion	early_stop_restart	8/20	40.0	39,143	1/7	7/9
react	self_consistency	8/20	40.0	42,329	2/7	6/9
act_only	checklist	7/20	35.0	38,778	0/7	6/9
react	baseline	7/20	35.0	39,141	1/7	6/9
act_only	early_stop_restart	7/20	35.0	41,825	2/7	5/9
reflexion	baseline	6/20	30.0	39,582	1/7	5/9
act_only	baseline	5/20	25.0	41,638	0/7	5/9
react	checklist	5/20	25.0	44,034	0/7	5/9
act_only	self_consistency	5/20	25.0	45,586	0/7	5/9
reflexion	checklist	4/20	20.0	45,406	0/7	4/9

Table 13: Detailed Reflexion mechanism ablation.

Condition	Pass	Avg. tokens
react_baseline	8/20	38,306
reflexion_baseline	8/20	37,230
reflexion_ep1_only	7/20	37,813
restart_without_reflection	5/20	46,284

- quixbugs_wrap

Scout solves 0/4 Mini-nightmare tasks even after the tool-call fix.

References

- Anonymous. Repairagent: An interactive benchmark for llm-based program repair. *arXiv preprint arXiv:2401.00000*, 2024.
- Anonymous. Inspectcoder: Benchmarking agentic code debugging under interactive constraints. *arXiv preprint arXiv:2501.00000*, 2025.
- Carlos Jimenez, John Yang, et al. Swe-bench+: Enhanced benchmarks for fixing real-world python bugs. In *Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- Noah Shinn, Ben Labash, and Ashwin Gopinath. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Tong Wu et al. Debugbench: Evaluating debugging capability of large language models. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- John Yang et al. Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.